

Bidaio Jagoy Translator: A Hybrid Client-Centric RAG System for Low-Resource Language Preservation

Tarsila Samille Santos da Silveira Silva¹

¹PPgSC/UFRN

tarsillasamile@gmail.com

Abstract. *The preservation of endangered languages constitutes a critical challenge in computational linguistics. This report presents the Bidaio Jagoy Translator, a specialized AI platform for the Land Dayak language, available at <https://bidaio.vercel.app/>. Utilizing a Hybrid Client-Centric Retrieval-Augmented Generation (RAG) architecture, the system shifts preliminary context orchestration to the client via Transformers.js. By synthesizing local glossary lookups, remote vector retrieval, and client-side neural re-ranking, it grounds Google Gemini 2.5 Flash, enabling high-fidelity translation driven by explicitly retrieved linguistic context. Preliminary evaluations indicate the viability of hybrid browser-based RAG architectures for creating resilient, low-resource language tools.*

Resumo. *A preservação de línguas ameaçadas de extinção constitui um desafio crítico na linguística computacional. Este artigo apresenta o Bidaio Jagoy Translator, uma plataforma de IA especializada para o dialeto Bidaio Jagoy (Land Dayak), disponível em <https://bidaio.vercel.app/>. Utilizando uma arquitetura Híbrida de Geração Aumentada por Recuperação (RAG) Centrada no Cliente, o sistema desloca a orquestração preliminar de contexto para o navegador via Transformers.js. Ao sintetizar buscas em glossários locais, recuperação vetorial remota e re-ranqueamento neural no lado do cliente, o sistema fundamenta o modelo Google Gemini 2.5 Flash, permitindo traduções de alta fidelidade impulsionadas por contexto linguístico explicitamente recuperado. Avaliações preliminares e o uso de um ciclo de correção pela comunidade indicam a viabilidade técnica e econômica de arquiteturas RAG híbridas para a criação de ferramentas resilientes em contextos de línguas com poucos recursos.*

1. Introduction

The rapid advancement of Natural Language Processing (NLP), driven by the Transformer architecture, has paradoxically widened the gap between global "high-resource" languages and indigenous "low-resource" vernaculars. Foundational models such as GPT-4 and Gemini are trained on datasets scraping the public web, where English, Mandarin, Spanish, and standard Malay constitute the vast majority of tokens.

Bidaio Jagoy is a distinct dialect. It possesses a vibrant oral tradition but a negligible digital footprint. Its presence is confined to scattered PDFs, academic papers from institutions like Universiti Malaysia Sarawak (UNIMAS), and community blogs

[UNIMAS 2025]. Consequently, when standard LLMs attempt to process Bidaio Jagoy, they exhibit catastrophic failure modes such as hallucination, code-switching to standard Malay, and grammatical drift [Campbell et al. 2019].

Retrieval-Augmented Generation (RAG) offers a superior paradigm for this use case. Instead of embedding knowledge into the model’s parameters (implicit memory), RAG retrieves relevant information at runtime (explicit memory) and feeds it to the model via the context window [InterSystems 2025]. The Bidaio Jagoy Translator utilizes this approach to function as a “teaching machine,” assembling a curriculum of vocabulary, sentence patterns, and grammar rules relevant to specific inputs. Beyond these high-resource initiatives, the Bidaio Jagoy Translator employs a Hybrid Client-Centric architecture, prioritizing cost efficiency and latency reduction. While our case study focuses on the Bidaio Jagoy dialect, the proposed RAG framework is **generalizable to any low-resource language context**, including regional dialects, creoles, and endangered languages globally, not limited to indigenous languages.

2. Related Work

Digital preservation of low-resource languages has gained traction with initiatives like *Masakhane* for African languages [Nekoto et al. 2020] and Meta’s *No Language Left Behind* (NLLB) [NLLB Team et al. 2022]. However, these approaches predominantly rely on massive server-side models (e.g., NLLB-200B), which require substantial infrastructure; furthermore, they rely on massive training corpora that are often unavailable for extremely low-resource dialects where data scarcity is the primary bottleneck.

In contrast, our work focuses on the benefits of RAG for domain-constrained translation, demonstrating that effective linguistic grounding can be achieved by offloading context preparation to the browser, significantly reducing the dependency on massive, server-side data preparation for community-led projects.

2.1. The Stack Overview

The system utilizes a modern serverless stack:

- **Frontend:** Vanilla JavaScript + HTML5.
- **Client-Side AI:** Transformers.js (@xenova/transformers) running in the browser [Hugging Face 2024].
- **Backend:** FastAPI (Python) on Vercel Serverless Functions.
- **AI Engine:** Google Gemini 2.5 Flash [Google AI for Developers 2025].
- **Database:** Supabase (PostgreSQL) with pgvector [Supabase 2021].

2.2. The “Fat Client” Philosophy

Using Transformers.js, the browser downloads a quantized embedding model (Xenova/all-MiniLM-L6-v2) and runs it locally. This allows for privacy-preserving local re-ranking and reduced server costs. A critical advantage of this hybrid architecture is its economic sustainability; by offloading embedding generation and re-ranking to the user’s device, the system minimizes the computational demand on the backend. This allows the application to operate within the free-tier limits of serverless platforms such as Vercel, effectively democratizing access to high-performance RAG tools for community-led preservation projects without the need for expensive GPU-enabled server clusters.

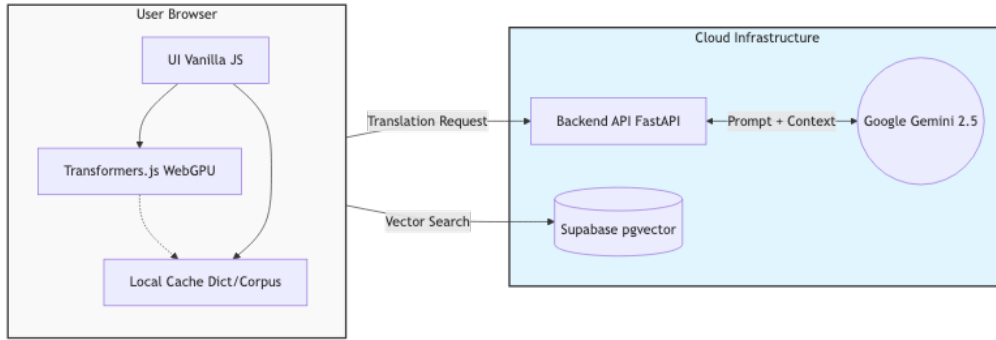


Figure 1. Hybrid Client-Centric System Architecture: Offloading embeddings to the browser.

3. Instructional Orchestration: RAG Prompt Engineering

The core of the Bidaio Jagoy Translator’s efficacy lies not merely in retrieval, but in the sophisticated orchestration of the Large Language Model (LLM) through prompt engineering. The system transforms from a simple retrieval engine into an expert linguistic synthesizer by constructing a multi-dimensional context.

3.1. Composition of the Instructional Context

The system builds a four-tiered prompt structure designed to ground the model’s reasoning. This is handled by a specialized `RAGPromptGenerator` class that manages the assembly of disparate data sources:

- **Context A (Lexical Anchor):** Direct glossary lookups for extracted keywords provide the base vocabulary. This prevents “lexical drift” where the LLM might use a more common but incorrect Malay word.
- **Context B (Structural Alignment):** Semantically similar sentence pairs are retrieved via a remote HNSW index on Supabase. These provide the LLM with the overall “vibe” and syntactic structure of the language.
- **Context C (Semantic re-ranking):** High-relevance usage examples for primary keywords are re-ranked locally. By calculating cosine similarity between the input embedding and candidate corpus sentences in the browser, the system ensures that the most semantically relevant nuances are presented first.
- **Context D (Grammatical Constraints):** Explicit rules that act as hard constraints (e.g., “Always use ‘piin’ for water in theological contexts”).

This tiered approach ensures that the LLM is not just predicting the next token, but performing a constrained synthesis based on linguistic “curriculum” injected at runtime. To handle this high-dimensional context, the system utilizes **Google Gemini 2.5 Flash** [Google Cloud 2025], selected for its 1-million-token context window [Data Studios 2024]. This enables **dense context injection**, where the model’s reasoning capabilities [LLM Stats 2024] are leveraged to resolve conflicts between corpus usage and explicit grammar rules.

3.2. Model Management and Caching

The selected embedding model, `all-MiniLM-L6-v2` (384-dim), is served in a 4-bit quantized ONNX format (~23MB). To ensure a smooth user experience, the system implements the **CacheStorage API**. Once the weights are retrieved during the first session,

Table 1. System Performance Metrics: Client-Side Retrieval and Synthesis

Stage	Metric	Mean Latency	Backend	Mechanism
Embedding Generation	Latency	~8.5ms - 20ms	WebGPU/WASM	Local Infer.
Vector Search	Recall@3	~150ms	Supabase	HNSW Search
Local Re-ranking	Precision@5	~12ms	WebGPU	Cos. Similarity
Prompt Assembly	Context Size	~2,500 tokens	JavaScript	Context Merge

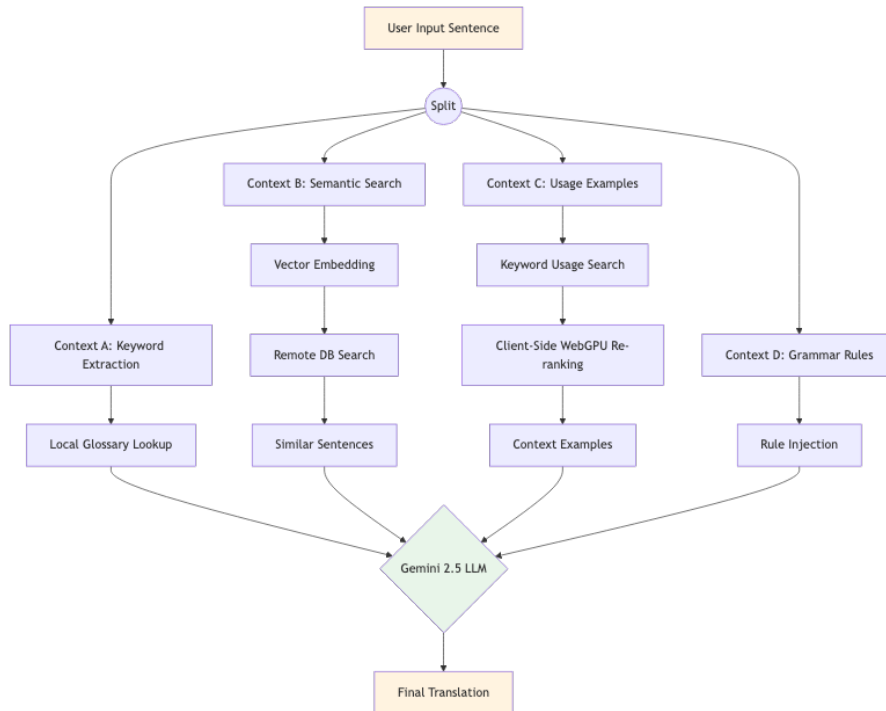
subsequent launches occur with zero network overhead for the AI runtime, significantly improving the user experience for recurring community members.

4. Context Assembly and Retrieval Strategy

The core innovation is the Multi-Staged Context Assembly pipeline.

4.1. The Hybrid Retrieval Pipeline

1. **Keyword Extraction:** Identifies semantic anchors (e.g., "water", "flows").
2. **Context A - Direct Glossary Lookup:** Queries local JSON dictionary for 1:1 translations.
3. **Context B - Semantic Vector Search:** Queries Supabase HNSW index [Supabase 2023] for similar sentence pairs.
4. **Context C - Contextual Usage Examples:** Use client-side embeddings to re-rank local corpus candidates, handling polysemy without server round-trips.
5. **Context D - Grammatical Constitution:** Injects explicit rules (e.g., Intensifier Placement).

**Figure 2. The 4-Stage Context Assembly Pipeline.**

4.2. Client-Side Re-ranking Algorithm

A key novelty is the semantic filtering performed in the browser to select the most relevant "Context C" examples. The system calculates the cosine similarity between the input embedding vector (A) and a candidate corpus vector (B) using the following formula:

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (1)$$

This operation, performed via WebGPU/WASM in the browser, ensures that the context injected into the LLM is optimized for the specific semantic nuances of the user's input.

Listing 1. Client-Side Cosine Similarity Re-ranking

```
def get_keyword_usage_context(text, keywords):
    input_embedding = generate_embedding(text) # Local WebGPU
    candidates = fetch_local_candidates(keywords)

    scored = []
    for cand in candidates:
        cand_embedding = generate_embedding(cand.target)
        score = cosine_similarity(input_embedding, cand_embedding)
        scored.append((score, cand))

    # Select top 3 most semantically similar examples
    top_results = sort_descending(scored)[:3]
    return format_for_prompt(top_results)
```

4.3. Structured "Chain-of-Thought" Prompting

The system constructs a structured prompt forcing the LLM to follow a specific cognitive process: Analysis $\rightarrow$ Vocabulary Selection $\rightarrow$ Synthesis $\rightarrow$ Output. This "Chain-of-Thought" (CoT) approach is critical for low-resource translation, as it forces the model to deliberate on the provided RAG context before committing to a translation.

The instructions strictly mandate a 3-step reasoning block:

1. **Deconstruction:** Identifying the grammatical structure from Context D and B.
2. **Mapping:** Selecting vocabulary from Context A and C, explicitly listing English-to-Bidaio mappings.
3. **Synthesis:** Assembling the final sentence according to the identified rules.

Listing 2. JSON structure for Chain-of-Thought prompting

```
{
  "reasoning": "1. Analyze: Input is SVO... 2. Map: 'friend' -> 'dayung' ... 3. Synthesize ...",
  "translation": "Final Bidaio text",
  "confidence_score": 5
}
```

The system's knowledge base relies on a curated parallel corpus derived from New Testament verses provided by community translators, aligned with a English reference text. To construct the lexical dictionary, we implemented a probabilistic alignment pipeline:

1. **Alignment Algorithms:** We utilized IBM Model 2 for positional alignment. This choice was dictated by its **low computational overhead** and its proven efficacy in scenarios with **extremely small corpora**, where more complex neural aligners tend to overfit or require prohibitive training time. Specifically, IBM Model 2

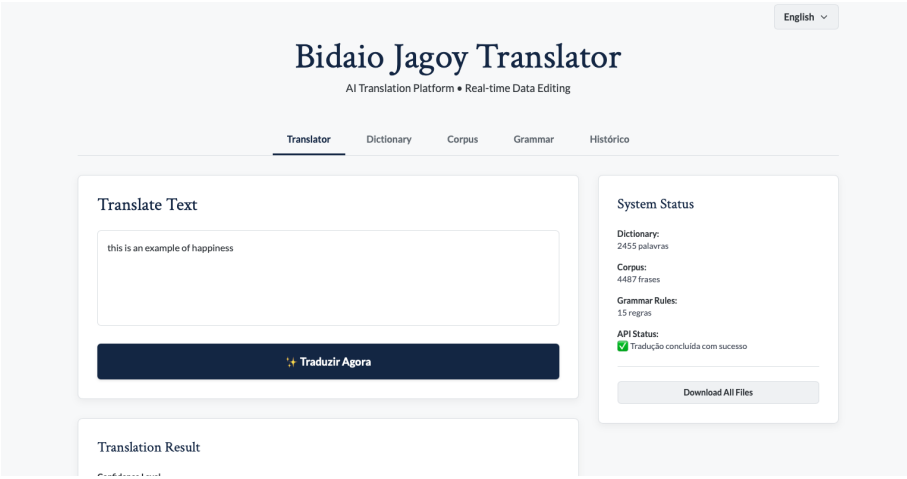


Figure 3. The Bidaio Jagoy Translator Interface: Input section and system status dashboard.

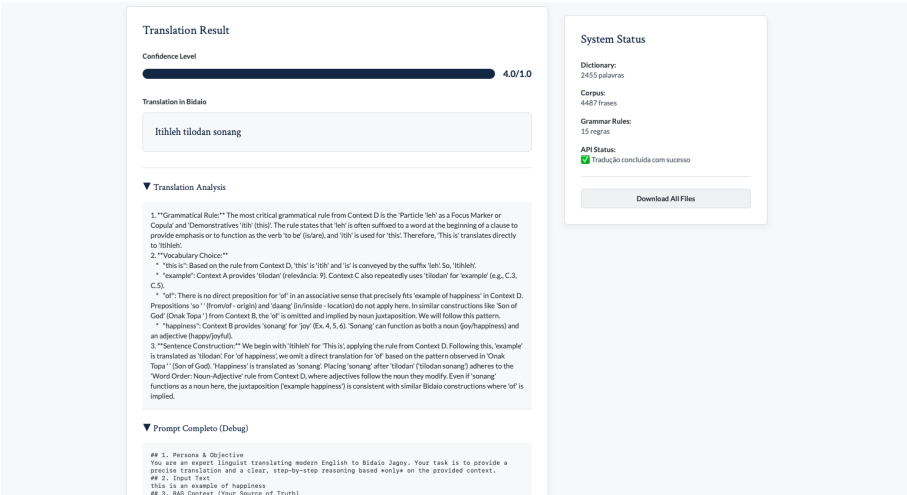


Figure 4. Linguistic Analysis Module: The system’s Chain-of-Thought reasoning, showing grammatical rule application and vocabulary mapping for the input ”this is an example of happiness”.

- treats word alignment as a latent variable problem with a simplified distance-based distortion model, which remains robust even when the parallel data consists of only a few thousand sentence pairs.
2. **Heuristic Refinement:** Candidate pairs underwent reclassification based on symmetry scores and cognate identification.
 3. **Filtering:** Only the top 50% of statistically significant word pairs were retained to ensure high precision. The final production corpus comprises 4,487 aligned sentence pairs and a lexical dictionary of 2,455 entries.

Furthermore, 15 core grammatical rules (e.g., Nominal Predication, Aspectual Markers) were extracted through an iterative Zero-Shot prompting process on Google Gemini. These rules are not treated as a static ”black box”; rather, they represent **exposed observational patterns** derived from the corpus. A critical feature of the Bidaio Jagoy Translator is that these rules, along with word alignments, are transparently displayed to

the user. This approach ensures "Human-in-the-loop" accountability, where any linguistic inaccuracy can be identified and corrected by native speakers, turning the system into a dynamic, community-vetted archive. This curated dataset serves as the foundation for the experimental evaluation discussed in the following section.

5. Experimental Evaluation and Discussion

5.1. Latency Breakdowns

The "Hybrid Client-Centric" model introduces a distributed latency profile:

- **Client Embeddings (WebGPU):** ~20ms. This negligible cost validates the feasibility of offloading compute to consumer devices.
- **Supabase Vector Search:** ~150ms (HNSW index).
- **LLM Inference (Gemini):** ~5s. This remains the dominant factor.

The total round-trip time averages 6s. While higher than cached translation services, this latency is acceptable for the project's primary goal: high-precision educational translation.

5.2. Community-Driven Correction Loop

A robust evaluation of translation quality in low-resource settings requires native speaker validation. The Bidaio Jagoy Translator implements an integrated *History and Correction Module*. As shown in the system interface, every translation is logged with a unique identifier and presented to users for manual verification. This mechanism allows the community to:

- **Validate:** Confirm the accuracy of AI-generated translations.
- **Correct:** Provide authoritative manual edits for "unedited" entries.
- **Annotate:** Flags errors which are then used to update the "Context D" (Grammar Rules) and "Context A" (Glossary).

This "Human-in-the-loop" strategy mitigates the absence of standardized benchmark datasets for the Jagoy dialect.

5.3. The System Prompt

To orchestrate the RAG context, we use a structured prompt that strictly enforces the use of retrieved data. A simplified version of the system prompt is shown below:

Listing 3. The Hybrid RAG System Prompt

```
You are a Bidaio Jagoy Translator. Use the following context:

[Context A: Dictionary Matches]
{dictionary_entries}

[Context B: Parallel Corpus Matches]
{corpus_matches}

[Context C: Grammar Rules]
{grammar_rules}

Task: Translate the following English text to Bidaio Jagoy:
"{user_input}"

Instructions:
1. Prioritize Context B for phrasing.
2. Use Context A for specific vocabulary.
3. Apply Context C to correct sentence structure.
4. Output specific JSON format with "analysis", "draft", and "final_translation".
```

5.4. Qualitative Case Study

To evaluate linguistic fidelity, we analyze real system outputs from the community validation log (Validation ID: 57 and 54).

- **Complex Theological Query:**

- *Input:* "He will command his angels concerning you, and they will lift you up in their hands, so that you will not strike your foot against a stone."
- *Bidaio Output:* "**Topa' ya' lubang malaikat-malaikat-E nulis Mu'u, duoh odop-e de' muno' ya' Nyamut Mu'u daang tongan-e, sape Mu'u ya' di'-e koming po Mu'u salah obuah batuh.**"
- *Analysis:* The system correctly handles the complex Subject-Verb-Object (SVO) structure and maps "God" (implied context) to "Topa'", "angels" to "malaikat-malaikat-E" (with possessive suffix), and "lift up" to "Nyamut".

- **Idiomatic/Imperative phrasing:**

- *Input:* "tell these stones to become bread."
- *Bidaio Output:* "**Oda'leh batuh-batuh itih jadi pinguman.**"
- *Analysis:* The system correctly identifies "stones" as plural "batuh-batuh" and selects "pinguman" (food/sustenance) for bread in this context, demonstrating semantic awareness beyond literal word-for-word translation.

These examples demonstrate the system's ability to ground the LLM in specific dialectal vocabulary and retain grammatical coherence for long-tail languages.

5.5. Error Analysis and Fallback Mechanisms

Crucial to the scientific evaluation of the Bidaio Jagoy Translator is the analysis of its failure modes. In cases where the RAG context fails to provide a specific Bidaio Jagoy term (out-of-vocabulary), the system implements a **Graceful Degradation Strategy** via an Indonesian pivot:

- **Example Failure:**

- *Input:* "...the sound of a thousand drums."
- *Bidaio Output:* "...okug (??? — 'seribu') tambur."
- *Analysis:* Here, the term "thousand" was missing from both Context A and Context B. instead of hallucinating a random Bidaio-sounding word, the system followed the system prompt instruction to output the Indonesian equivalent flagged as (???).

This explicit signaling of uncertainty is a key strength. It prevents the propagation of "silent errors" and directly informs the community-driven correction loop. Native speakers can quickly identify these flags in the translation logs and provide the correct Bidaio term (e.g., "ribu"), which is then used to update the central glossary, closing the RAG knowledge gap for future users.

6. Conclusion

The Bidaio Jagoy Translator represents a synthesis of advanced neural search and traditional rule-based linguistics. By leveraging Transformers.js, the system democratizes access to advanced RAG tools for endangered language preservation, decoupling it from expensive server-side GPU clusters.

Importantly, while demonstrated through Bidaio Jagoy, this hybrid client-centric RAG architecture is **applicable to any low-resource language scenario** — including regional dialects (e.g., Venetian, Sicilian), creoles (e.g., Tok Pisin), minority languages (e.g., Sorbian, Romansh), and any language on the UNESCO Red List. The minimal requirements are: (1) a small parallel corpus ($\sim 2,000+$ sentence pairs), (2) community involvement for validation, and (3) willingness to operate within a browser-based deployment model.

Future work will explore a "Fully Offline Mode" by integrating small-scale LLMs (e.g., Gemma 2B or TinyLlama) directly into the browser. This would eliminate the dependency on cloud connectivity entirely, creating a truly resilient digital archive for the Bidaio community.

References

- Campbell, Y. M. et al. (2019). Intensifiers in bidayuh bau-jagoy. *Issues in Language Studies*. Accessed: 2025-12-22.
- Data Studios (2024). Google gemini 2.5 flash: Context window and token limits. Accessed: 2025-12-22.
- Google AI for Developers (2025). *Gemini API Changelog*. Accessed: 2025-12-22.
- Google Cloud (2025). Gemini 2.5 flash on vertex ai. Accessed: 2025-12-22.
- Hugging Face (2024). Transformers.js. Accessed: 2025-12-22.
- InterSystems (2025). Rag vs fine-tuning vs prompt engineering. Accessed: 2025-12-22.
- LLM Stats (2024). Gemini flash series: Performance benchmarks and infrastructure costs. Accessed: 2025-12-22.
- Nekoto, W. et al. (2020). Participatory research for low-resource machine translation: A case study in african languages. In *Findings of EMNLP 2020*. Accessed: 2025-12-22.
- NLLB Team et al. (2022). No language left behind: Scaling human-centered machine translation. *arXiv preprint arXiv:2207.04672*. Accessed: 2025-12-22.
- Supabase (2021). *pgvector: Embeddings and vector similarity*. Accessed: 2025-12-22.
- Supabase (2023). pgvector: Faster semantic search with hnsw indexes. Accessed: 2025-12-22.
- UNIMAS (2025). Unimas institutional repository: Philology. linguistics. Accessed: 2025-12-22.

Reproducibility and Availability

The complete system is publicly available to support reproducibility and enable adaptation to other low-resource languages:

- **Production System:** <https://bidaio.vercel.app/>
- **Source Code:** <https://github.com/TarsilaSamille/bidaio>
- **Technology Stack:** FastAPI (backend), Vanilla JavaScript (frontend), Transformers.js (client-side AI), Supabase (vector database)
- **Dataset:** Available upon request to the Bidaio community for research purposes

A. Key System Specifications

Table 2. Key System Specifications			
Component	Technology	Specification	Purpose
AI Orchestrator	Gemini 2.5 Flash	1M Context Window	RAG Synthesis
Prompt Strategy	Reactive CoT	4-Tiered Context	Linguistic Grounding
Local Embedder	all-MiniLM-L6-v2	384-dim (WebGPU)	Client-side RAG
Vector Store	Supabase HNSW	Cosine Distance	Global Retrieval
Linguistic Ref.	Bidaio Jagoy	SVO Dialect	Target Language